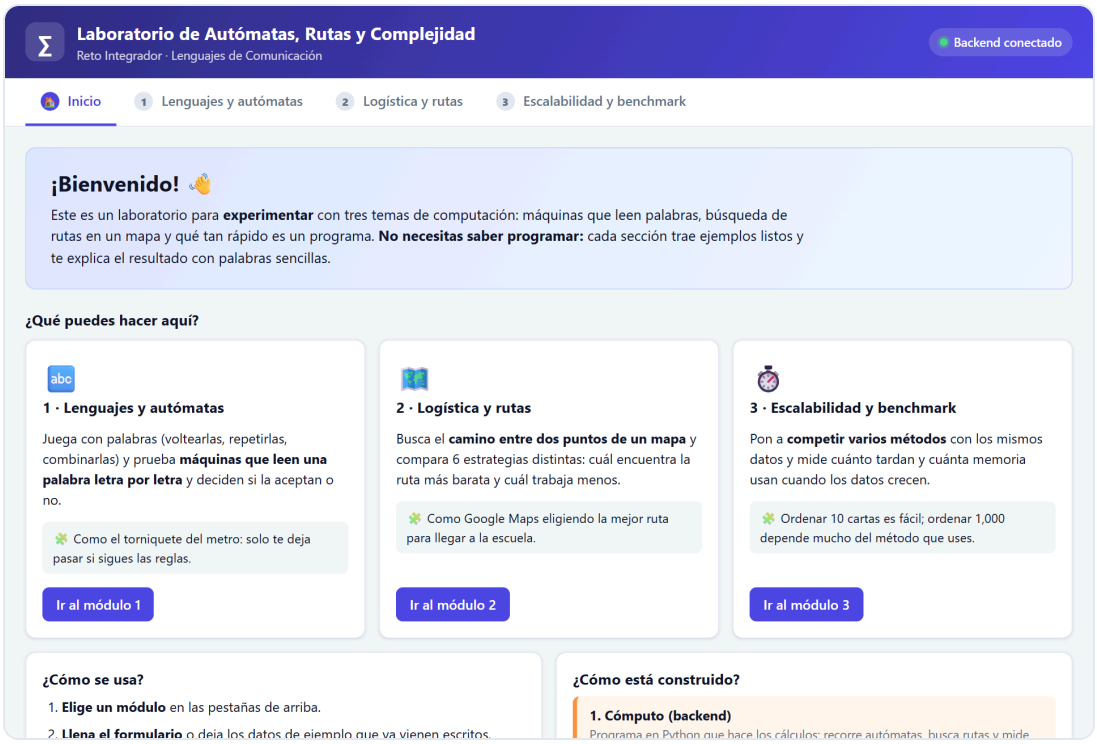
Laboratorio de Autómatas, Rutas y Complejidad

Manual de Usuario



Reto Integrador · Lenguajes de Comunicación

Versión 1.0 · Octubre 2026

Contenido

1. Introducción
2. Cómo abrir el laboratorio
    1. Requisitos
    2. Arrancar el backend y el frontend
    3. Indicador de conexión
3. Pantalla de inicio y navegación
4. Módulo 1 · Lenguajes y autómatas
    1. Operaciones con cadenas
    2. Operaciones con lenguajes
    3. Simulador de autómatas
    4. Reproductor paso a paso
    5. Probar varias palabras a la vez
    6. AFND y AFND-ε
5. Módulo 2 · Logística y rutas
    1. Elegir el mapa (grafo)
    2. Comparar rutas
    3. Leer la tabla y la gráfica
    4. ¿Hasta dónde llego? (alcance)
6. Módulo 3 · Escalabilidad y benchmark
    1. Competencia de ordenamiento
    2. Agente viajero
    3. ¿Qué pasa si crece? (P vs NP)
7. Mensajes de error
8. Preguntas frecuentes
9. Glosario

1. Introducción

El **Laboratorio de Autómatas, Rutas y Complejidad** es una aplicación web para que estudiantes de nuevo ingreso **experimenten** con tres temas de computación sin necesidad de saber programar:

Módulo	Qué puedes hacer	Ejemplo de la vida diaria
1 · Lenguajes y autómatas	Aplicar operaciones a palabras y a conjuntos de palabras, y probar máquinas (autómatas) que leen una palabra letra por letra y deciden si la aceptan.	El torniquete del metro: solo te deja pasar si sigues las reglas.
2 · Logística y rutas	Buscar el camino entre dos puntos de un mapa con 6 estrategias distintas y compararlas.	Una app de mapas eligiendo la mejor ruta a la escuela.
3 · Escalabilidad y benchmark	Poner a competir algoritmos con los mismos datos y medir cuánto tardan y cuánta memoria usan cuando los datos crecen.	Ordenar 10 cartas es fácil; ordenar 1,000 depende mucho del método.

Cómo se usa cualquier pantalla

Todas las secciones funcionan igual:

1. **Elige un módulo** en las pestañas de arriba.
2. **Lee el recuadro amarillo** 📘 **¿Qué es esto?**: explica el tema con palabras sencillas, una analogía y los pasos a seguir.
3. **Llena el formulario** o deja los datos de ejemplo que ya vienen escritos.
4. Presiona el **botón morado** (por ejemplo Evaluar cadena).
5. Revisa el resultado y lee el recuadro azul 💡 **¿Qué pasó?**, donde se explica el resultado y su base teórica.

> 💡 Junto a cada tabla y gráfica hay una nota 📖 **Cómo leer esto**. Al final de cada resultado puedes abrir **Ver la respuesta JSON del backend** para ver exactamente lo que respondió el servidor.

2. Cómo abrir el laboratorio

2.1 Requisitos

- Python 3.10 o superior.
- Un navegador moderno (Chrome, Edge o Firefox).
- Conexión a internet para que se dibujen las gráficas (la librería Chart.js se descarga de internet).

2.2 Arrancar el backend y el frontend

El laboratorio tiene dos partes que deben estar corriendo al mismo tiempo, cada una en su propia terminal:

Abre dos terminales en la raíz del repositorio (la carpeta que contiene `requirements.txt`):

```
# Terminal 1 · backend (hace los cálculos), puerto 8000
pip install -r requirements.txt      # solo la primera vez
uvicorn app.main:app --reload

# Terminal 2 · frontend (la página), puerto 5500
cd frontend
python -m http.server 5500
```

Después abre en el navegador: **http://localhost:5500**

> ⚠️ No abras `index.html` con doble clic: el navegador bloquea los módulos de JavaScript desde `file://` y la página no funciona. Usa siempre la dirección `http://localhost:5500`.

2.3 Indicador de conexión

En la esquina superior derecha aparece el estado del backend:

- 🟢 **Backend conectado**: todo está listo.
- 🔴 **Backend sin conexión**: el backend no está corriendo o está en otra dirección.

Si el backend corre en otra dirección o puerto, haz clic en el indicador, escribe la URL y presiona Guardar y reconectar. La página revisa la conexión automáticamente cada 15 segundos.



Figura 1. Panel de conexión: aquí se cambia la URL del backend.

3. Pantalla de inicio y navegación

Al abrir el laboratorio aparece la pestaña 🏠 **Inicio**. Desde aquí puedes ir a cualquier módulo con las **pestañas superiores** o con los botones Ir al módulo 1, Ir al módulo 2 y Ir al módulo 3.

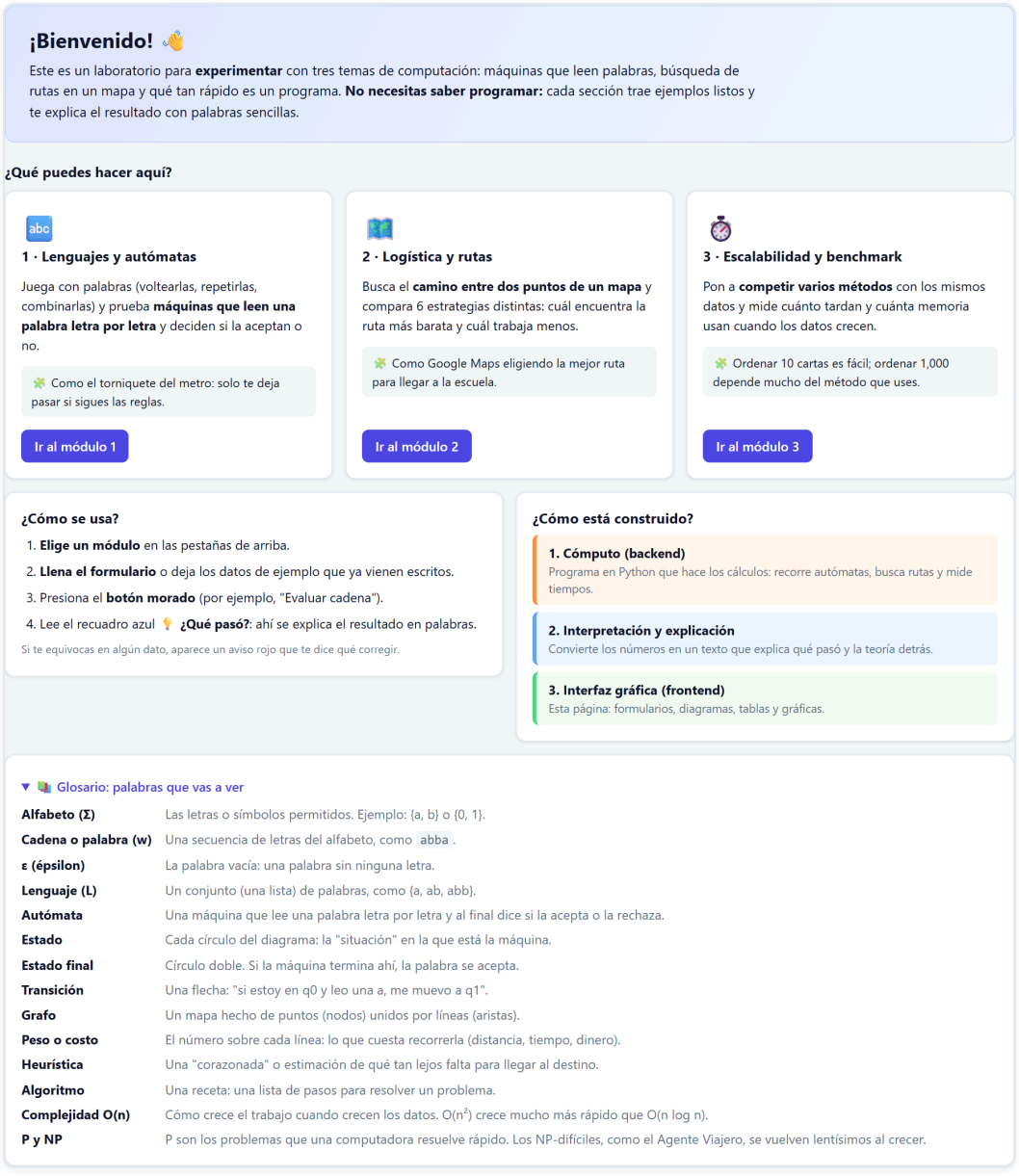


Figura 2. Pantalla de inicio: tarjetas de los módulos, cómo se usa, cómo está construido y glosario.

La pantalla de inicio contiene:

- **¿Qué puedes hacer aquí?**: una tarjeta por módulo con un ejemplo de la vida diaria.
- **¿Cómo se usa?**: los 4 pasos básicos.
- **¿Cómo está construido?**: las tres capas del sistema (cómputo, interpretación e interfaz).
- 📚 **Glosario**: haz clic para desplegar las definiciones de alfabeto, cadena, ε, autómata, grafo, heurística, complejidad, P y NP, etc.

4. Módulo 1 · Lenguajes y autómatas

Este módulo tiene tres sub-pestañas: **Cadenas**, **Lenguajes** y **Autómatas**.

4.1 Operaciones con cadenas

Una **cadena** (o palabra) es una fila de letras, como `abba`. Aquí le aplicas operaciones.



Figura 3. Formulario de operaciones con cadenas.

Campo	Para qué sirve
Palabra w	La palabra principal. Para la palabra vacía escribe `ε`.
Palabra v	Solo se usa para la concatenación (pegar w y v).
Número n	Cuántas veces repetir w (potencia), de 0 a 50.
Letras permitidas Σ	El alfabeto, separado por comas. Se usa para revisar si w ∈ Σ*.

Pasos:

1. Escribe la palabra en **w** (viene `abba` de ejemplo).
2. Marca las operaciones que quieras: longitud, inversa, concatenación, potencia, prefijos, sufijos, subcadenas, palíndromo, conteo de símbolos y pertenencia a Σ*.
3. Presiona Aplicar operaciones.

Aparece una **tarjeta por operación** con el resultado y una frase que lo explica. Cada tarjeta tiene un enlace ▶ **Teoría** para ver la definición formal.

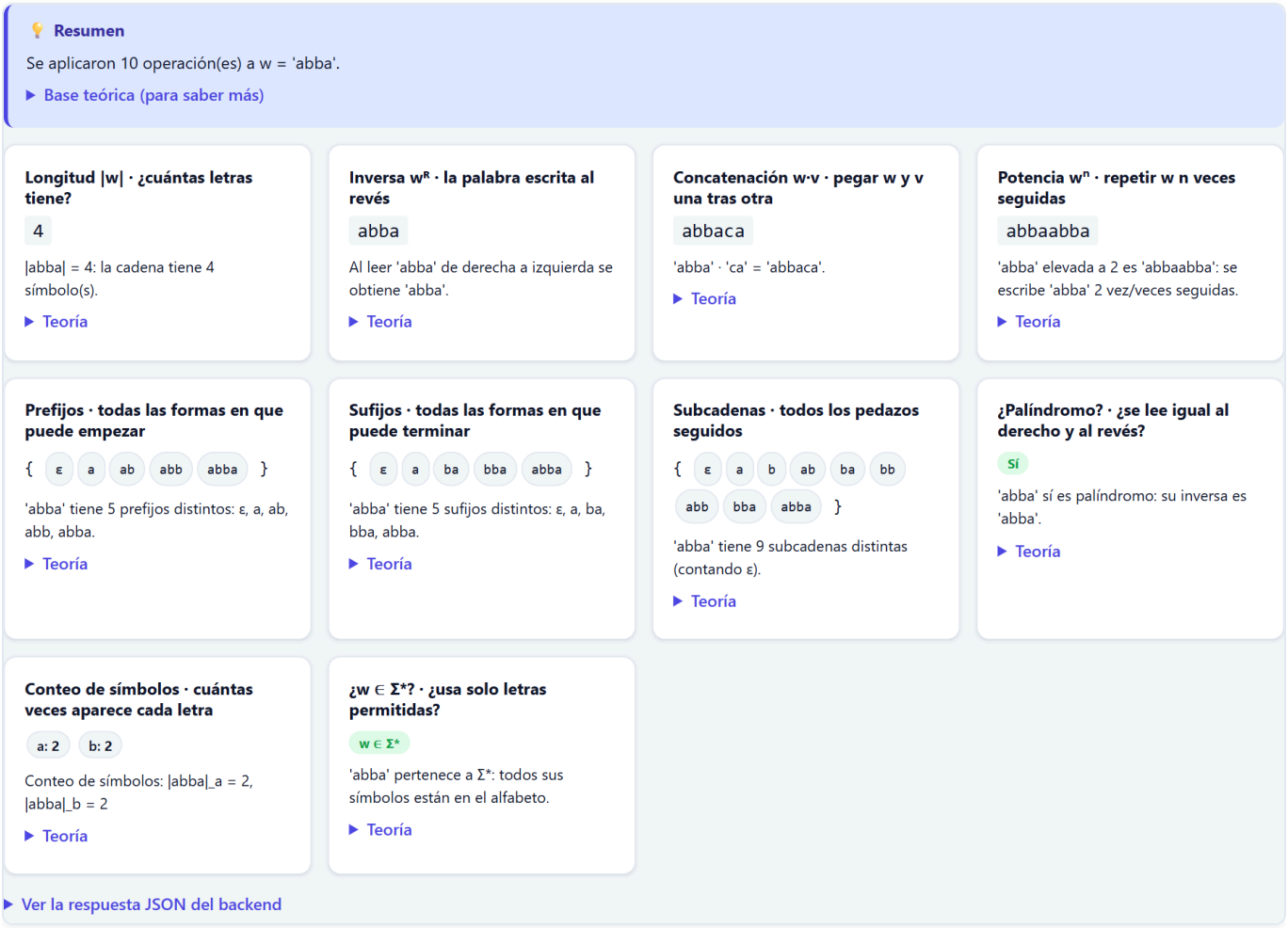


Figura 4. Resultado: una tarjeta por cada operación aplicada a «abba».

4.2 Operaciones con lenguajes

Un **lenguaje** es un conjunto de palabras, por ejemplo L = {a, ab}. Como son conjuntos, se pueden unir, intersectar, restar y combinar.



Figura 5. Formulario de operaciones con lenguajes.

1. Escribe las palabras de **L** y de **M** separadas por comas.
2. Escribe las **letras permitidas Σ** (se usan para el complemento) y el **número n** para la potencia L^n.
3. Mueve el deslizador **Largo máximo k**. L*, L^+ y el complemento tienen infinitas palabras, así que solo se muestran las de hasta k letras.
4. Marca las operaciones y presiona **Aplicar operaciones**.

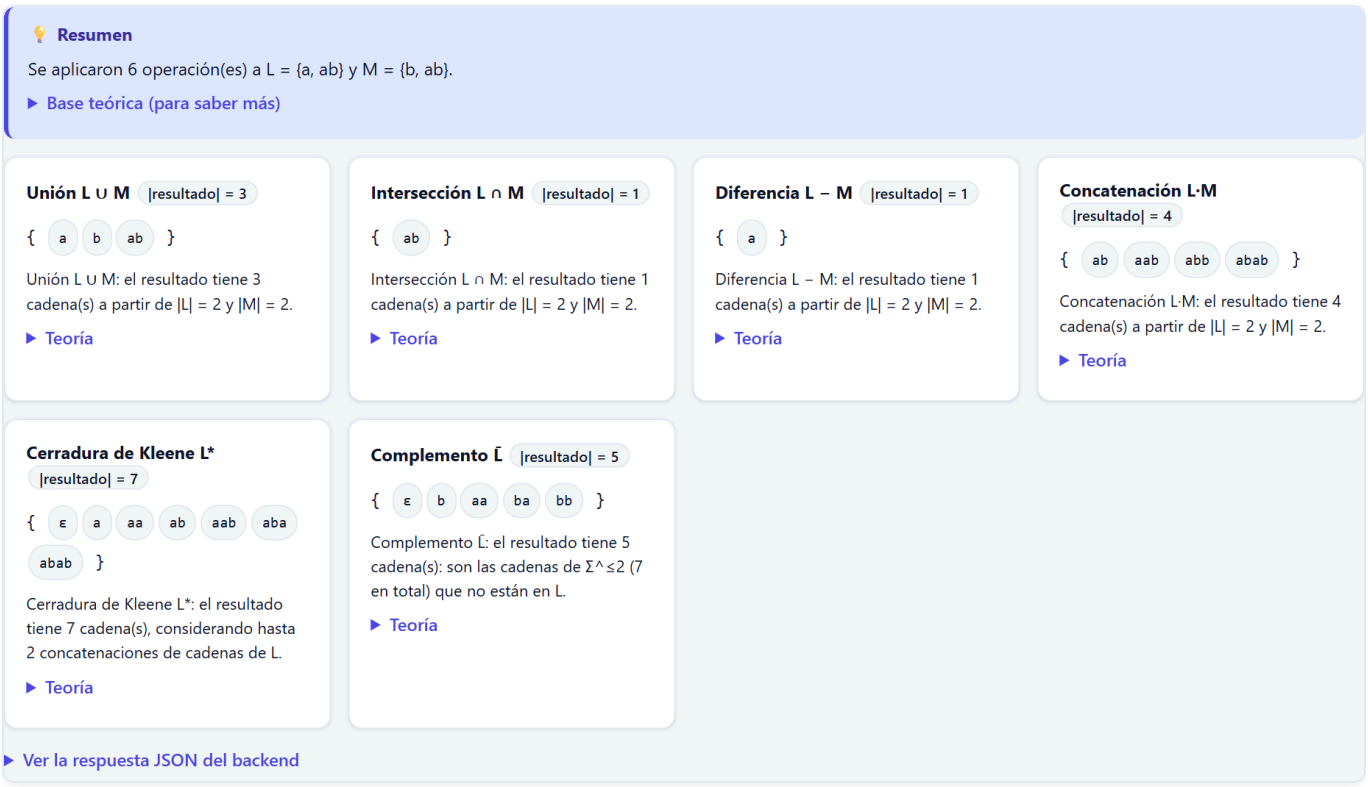


Figura 6. Cada conjunto resultado aparece con su cardinalidad (cuántas palabras tiene).

4.3 Simulador de autómatas

Un **autómata** es una máquina que lee una palabra **letra por letra**. Los círculos son **estados** y las flechas dicen a dónde moverse con cada letra. Si al terminar queda en un **círculo doble** (estado final), la palabra se **acepta**.

Si tienes dudas sobre los tipos de autómata, despliega **¿Cuál es la diferencia entre AFD, AFND y AFND-ε?**:

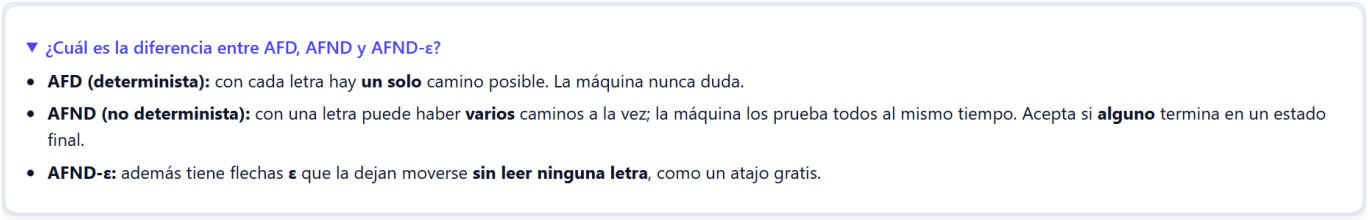


Figura 7. Explicación de los tres tipos de autómata.

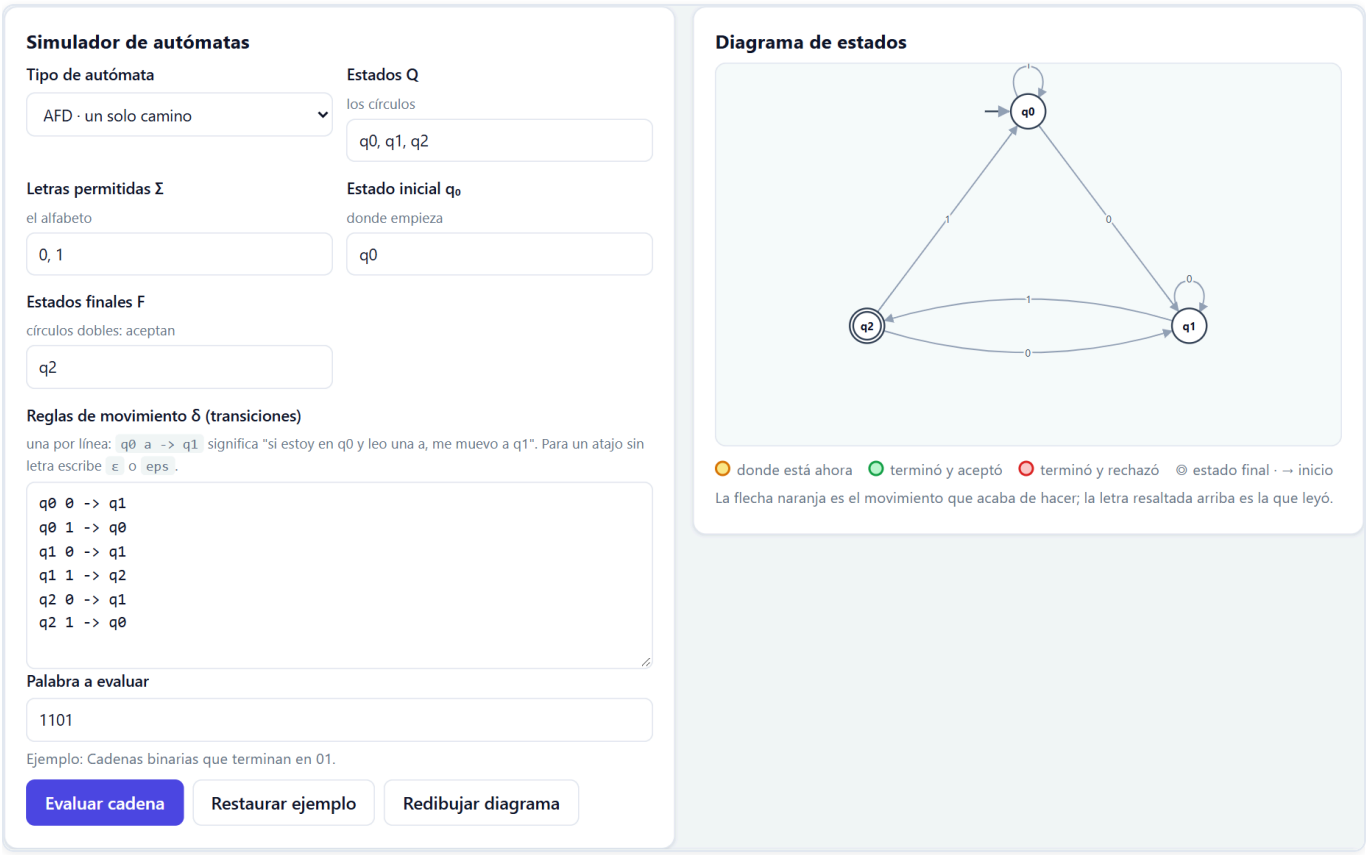


Figura 8. Editor del autómata (izquierda) y diagrama de estados (derecha).

Campo	Qué escribir	Ejemplo
Tipo de autómata	AFD, AFND o AFND-ε. Al cambiarlo se carga un ejemplo listo.	AFD · un solo camino
Estados Q	Los círculos, separados por comas.	`q0, q1, q2`
Letras permitidas Σ	El alfabeto (un carácter por símbolo).	`0, 1`
Estado inicial q_0	Donde empieza la máquina.	`q0`
Estados finales F	Los círculos dobles (aceptan).	`q2`

Reglas de movimiento δ	Una transición por línea: `estado letra -> destino`. En AFND puede haber varios destinos; en AFND-ε usa `ε` o `eps` como letra.	`q0 0 -> q1`
		`q0 a -> q0 q1`
		`q0 ε -> q1`

Palabra a evaluar	La palabra que leerá la máquina.	`1101`

Botones:

- Evaluar cadena: envía el autómata al backend y muestra el veredicto.
- Restaurar ejemplo: vuelve a cargar el ejemplo del tipo elegido.
- Redibujar diagrama: actualiza el dibujo después de editar las reglas, sin evaluar.

Al evaluar aparece el **veredicto** (verde si se acepta, rojo si se rechaza), la tabla **Recorrido paso a paso** y la explicación 💡 **¿Qué pasó?** con la complejidad.

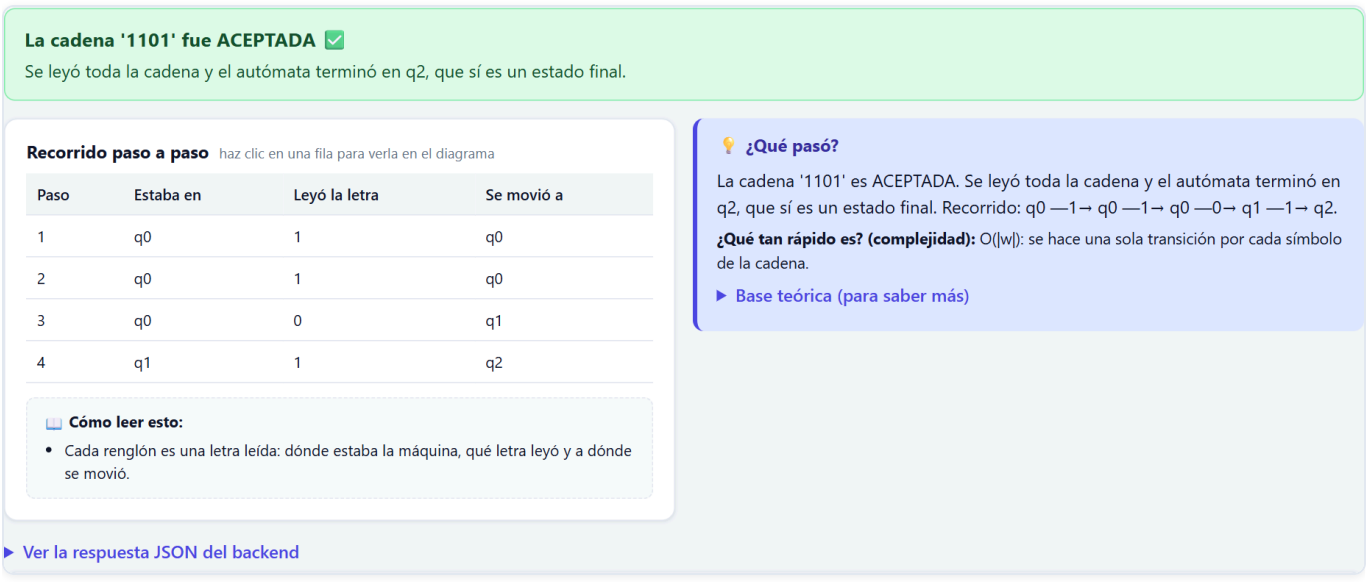


Figura 9. La cadena «1101» es aceptada por el AFD de ejemplo (cadenas binarias que terminan en 01).

4.4 Reproductor paso a paso

Después de evaluar, sobre el diagrama aparece la **cinta** con las letras de la palabra y, debajo, los controles del reproductor:

Botón	Acción
⏮	Ir al inicio (antes de leer la primera letra).
◀	Retroceder un paso.
▶ Reproducir	Animar todo el recorrido automáticamente (cambia a pausa mientras corre).
▶	Avanzar un paso.
⏭	Ir al final y ver el resultado.



Figura 10. Paso 3: la máquina lee la letra «0» (resaltada en la cinta) y pasa de q0 a q1 por la flecha naranja.

Colores del diagrama: **amarillo** = donde está ahora; **verde** = terminó y aceptó; **rojo** = terminó y rechazó. La **flecha naranja** es el último movimiento. También puedes hacer clic en una fila de la tabla **Recorrido paso a paso** para saltar a ese paso.

4.5 Probar varias palabras a la vez

Debajo del simulador está **Probar varias palabras a la vez**. Usa el mismo autómata de arriba.

1. Escribe una palabra por renglón (`ε` = palabra vacía).
2. Presiona **Probar todas**.
3. Observa cuáles se aceptan y cuáles no, y trata de descubrir la regla que sigue la máquina.

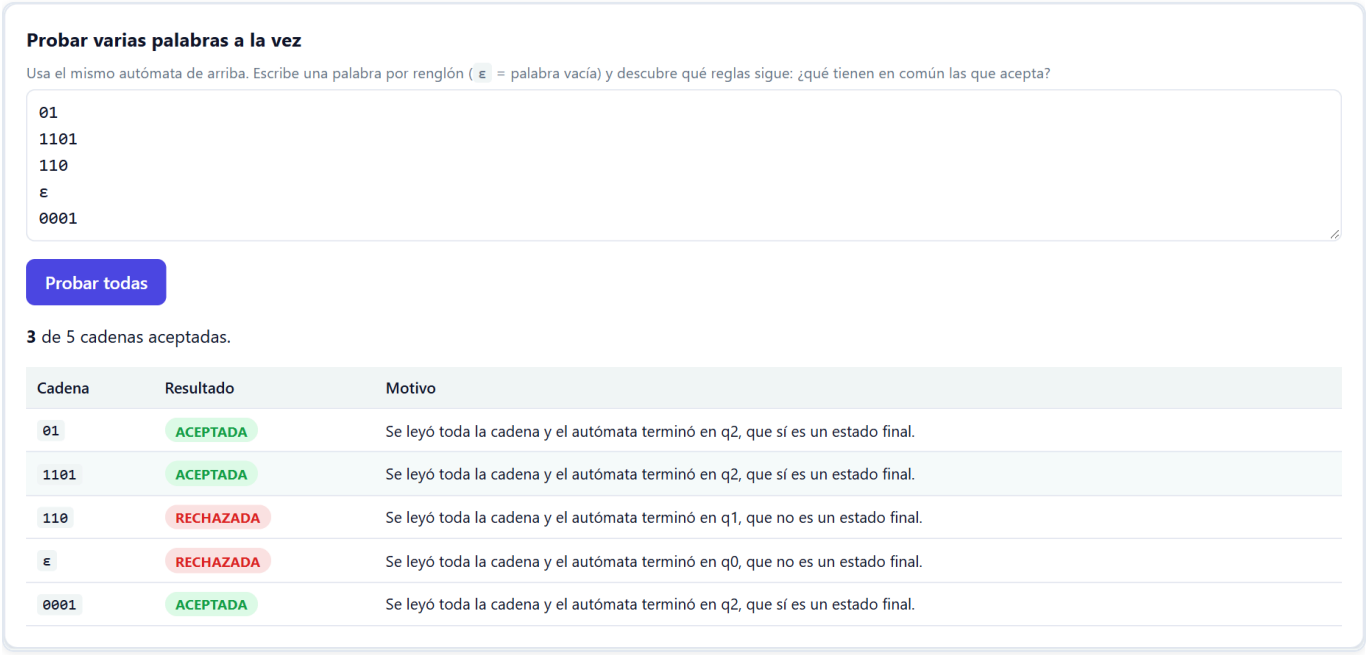


Figura 11. Resultado de probar cinco palabras con el AFD de ejemplo.

4.6 AFND y AFND-ε

En un **AFND** una letra puede llevar a varios estados a la vez, así que la máquina sigue todos los caminos al mismo tiempo y la palabra se acepta si **alguno** termina en un estado final. En un **AFND-ε** además hay flechas **ε** que permiten moverse sin leer letra (cerradura-ε). En estos tipos el diagrama resalta un **conjunto** de estados en cada paso.

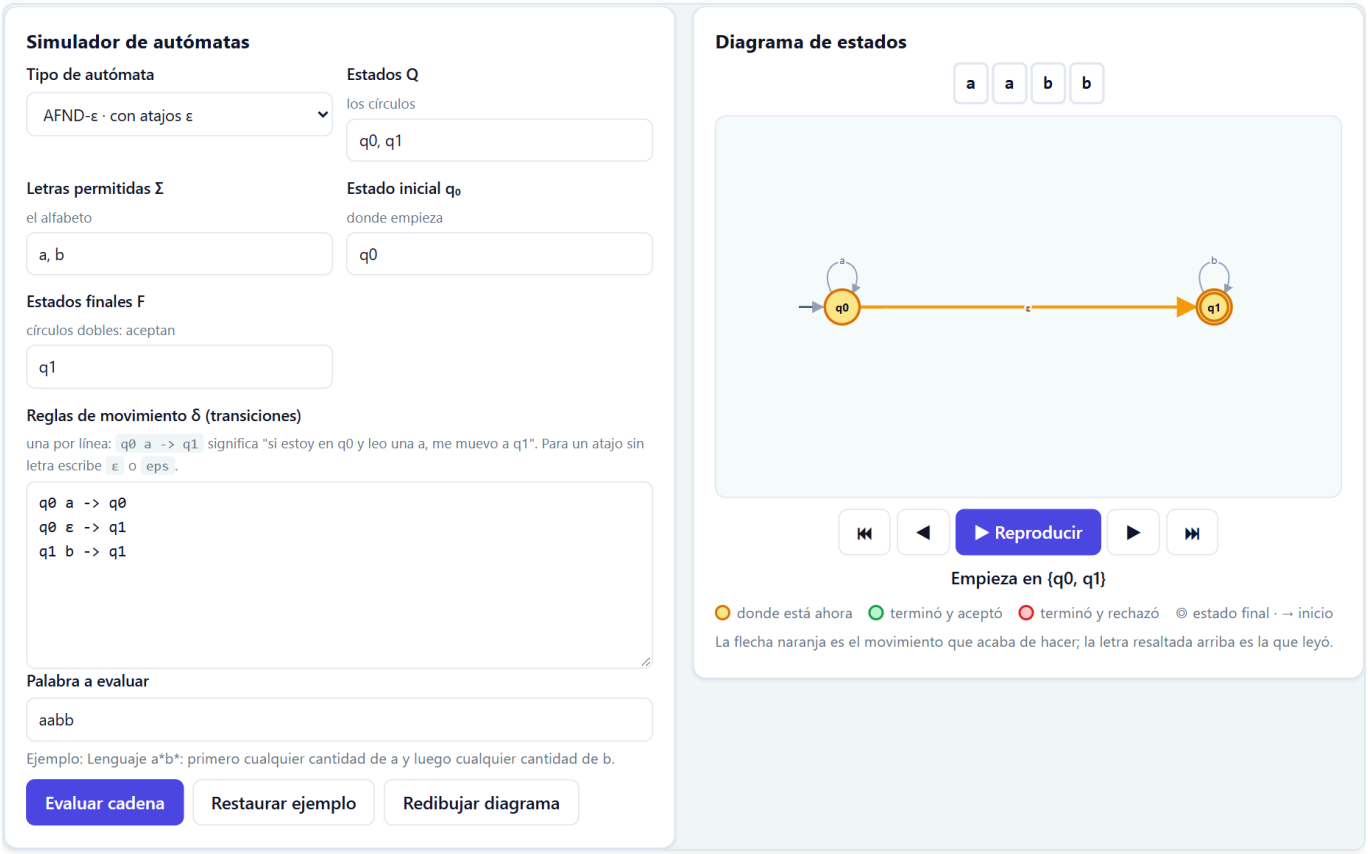


*Figura 12. AFND-ε de ejemplo (lenguaje a*b*). Desde el inicio la máquina ya está en {q0, q1} gracias a la transición ε.*



Figura 13. Traza del AFND-ε: en cada paso se muestran los conjuntos de estados y la cerradura-ε.

5. Módulo 2 · Logística y rutas

Un **grafo** es como un mapa: los círculos (**nodos**) son lugares, las líneas (**aristas**) son caminos y el número sobre cada línea es su **costo**. La pantalla se divide en los controles (izquierda) y **El mapa** (derecha).

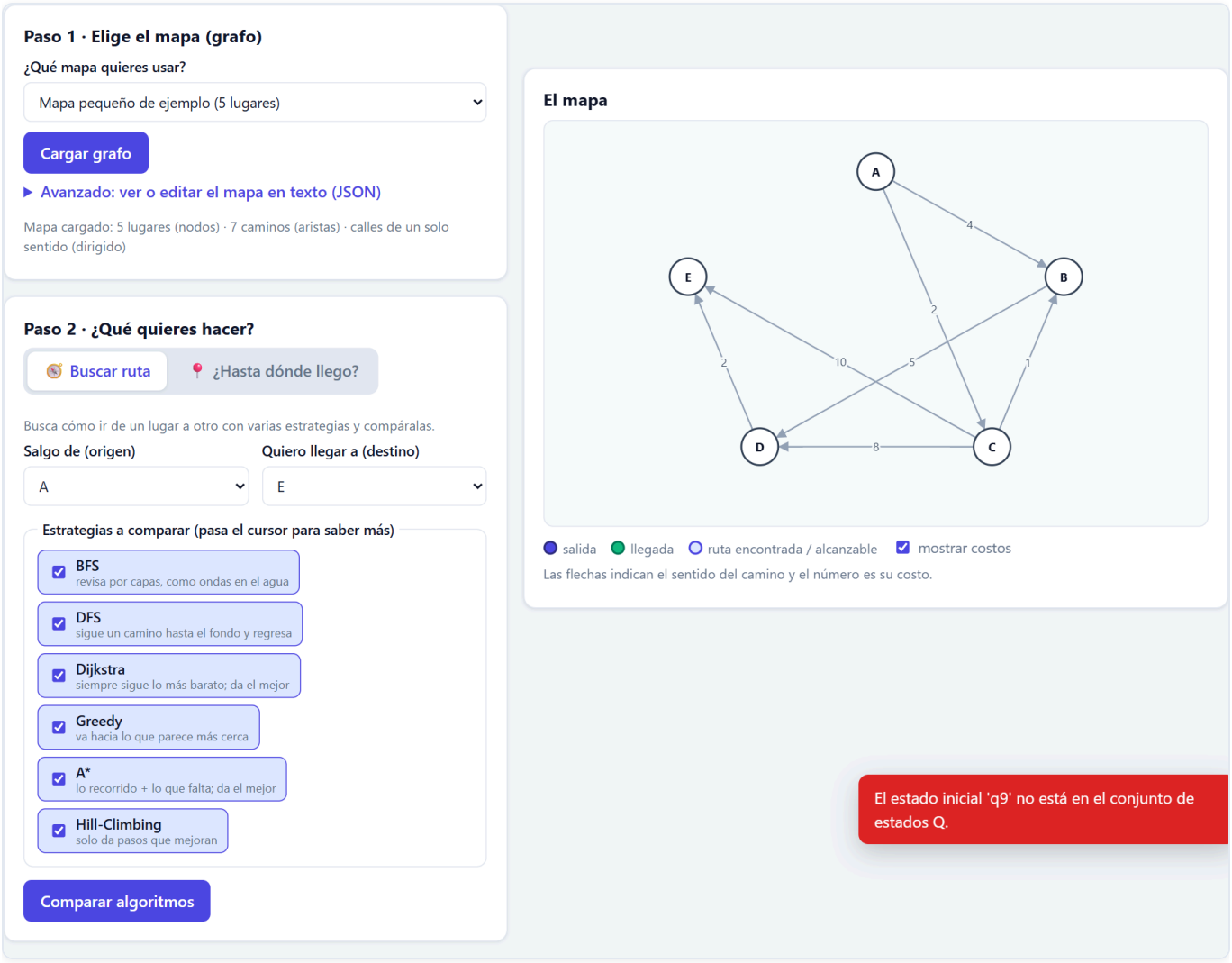


Figura 14. Pantalla del módulo 2 con el mapa pequeño de ejemplo cargado.

5.1 Paso 1 · Elegir el mapa (grafo)

En **¿Qué mapa quieres usar?** elige una opción y presiona Cargar grafo:

Opción	Descripción
Mapa pequeño de ejemplo	5 lugares, ideal para empezar.
Grafo del profesor · 10 / 25 / 50 lugares	Grafos dirigidos (calles de un solo sentido) y ponderados.
Generar uno al azar	Muestra los controles **Lugares (nodos)** (2 a 30), **Cantidad de caminos** (densidad de 0 a 1) y **Semilla** (el mismo número genera el mismo mapa).

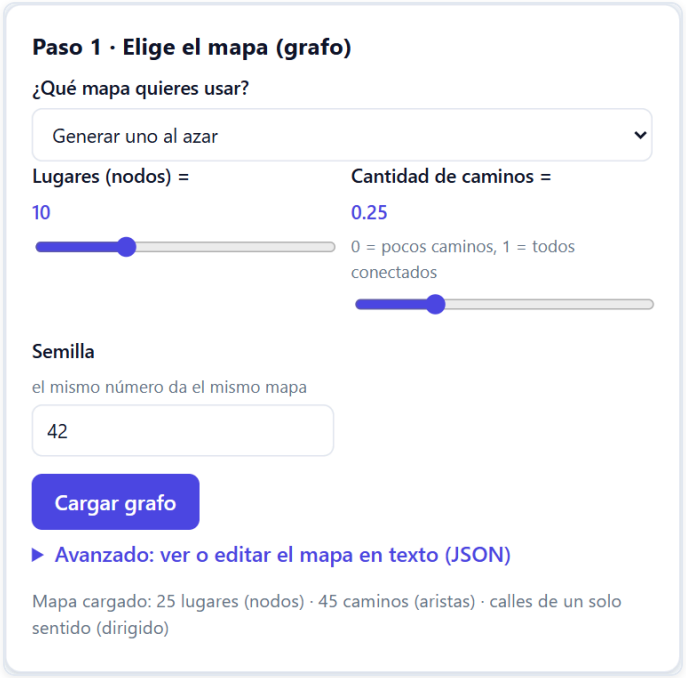


Figura 15. Opciones para generar un grafo al azar.

💡 En **Avanzado: ver o editar el mapa en texto (JSON)** puedes escribir tu propio grafo. `{"A": [["B", 15]]}` significa «de A sale un camino a B que cuesta 15». Marca **Calles de un solo sentido** si el grafo es dirigido y presiona **Aplicar cambios**.

5.2 Paso 2 · Comparar rutas (🧭 Buscar ruta)

1. En **Salgo de (origen)** y **Quiero llegar a (destino)** elige los lugares.
2. Marca las **estrategias a comparar**. Pasa el cursor sobre cada una para leer cómo funciona.
3. Presiona Comparar algoritmos.

Estrategia	Idea	¿Garantiza la ruta más barata?
BFS	Revisa por capas, como ondas en el agua. Da el camino con menos paradas.	No (sí la de menos paradas)
DFS	Sigue un camino hasta el fondo y regresa.	No
Dijkstra	Siempre sigue lo más barato acumulado.	**Sí**
Greedy	Va hacia lo que parece más cerca del destino.	No
A*	Lo recorrido + una estimación de lo que falta.	**Sí**
Hill-Climbing	Solo da pasos que mejoran; nunca retrocede.	No

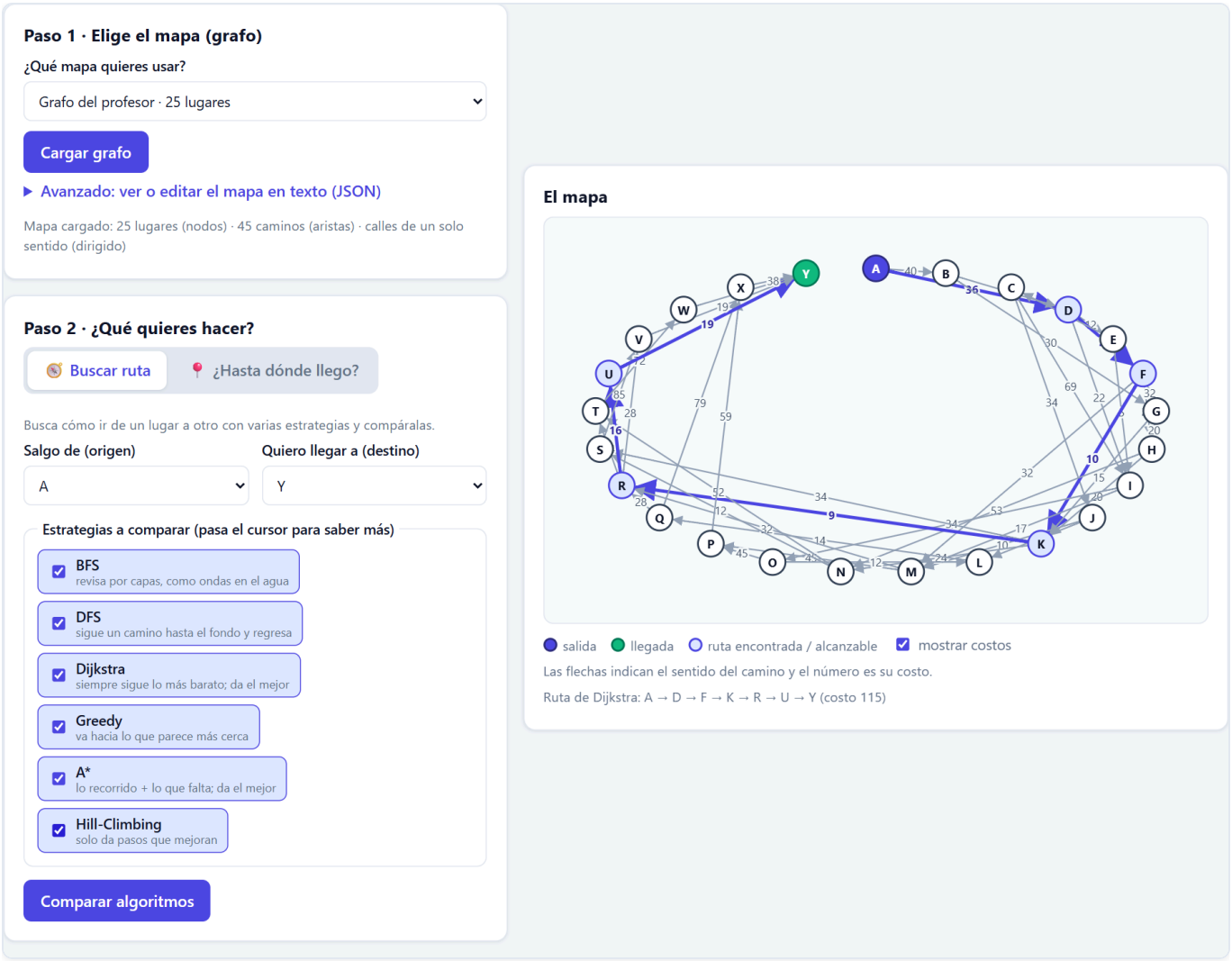


Figura 16. Grafo del profesor de 25 lugares, de A a Y. En el mapa se pinta la ruta del algoritmo seleccionado.

En el mapa: **morado fuerte** = salida, **verde** = llegada, **líneas moradas** = ruta encontrada. Desmarca **mostrar costos** si el mapa se ve muy cargado.

5.3 Leer la tabla y la gráfica

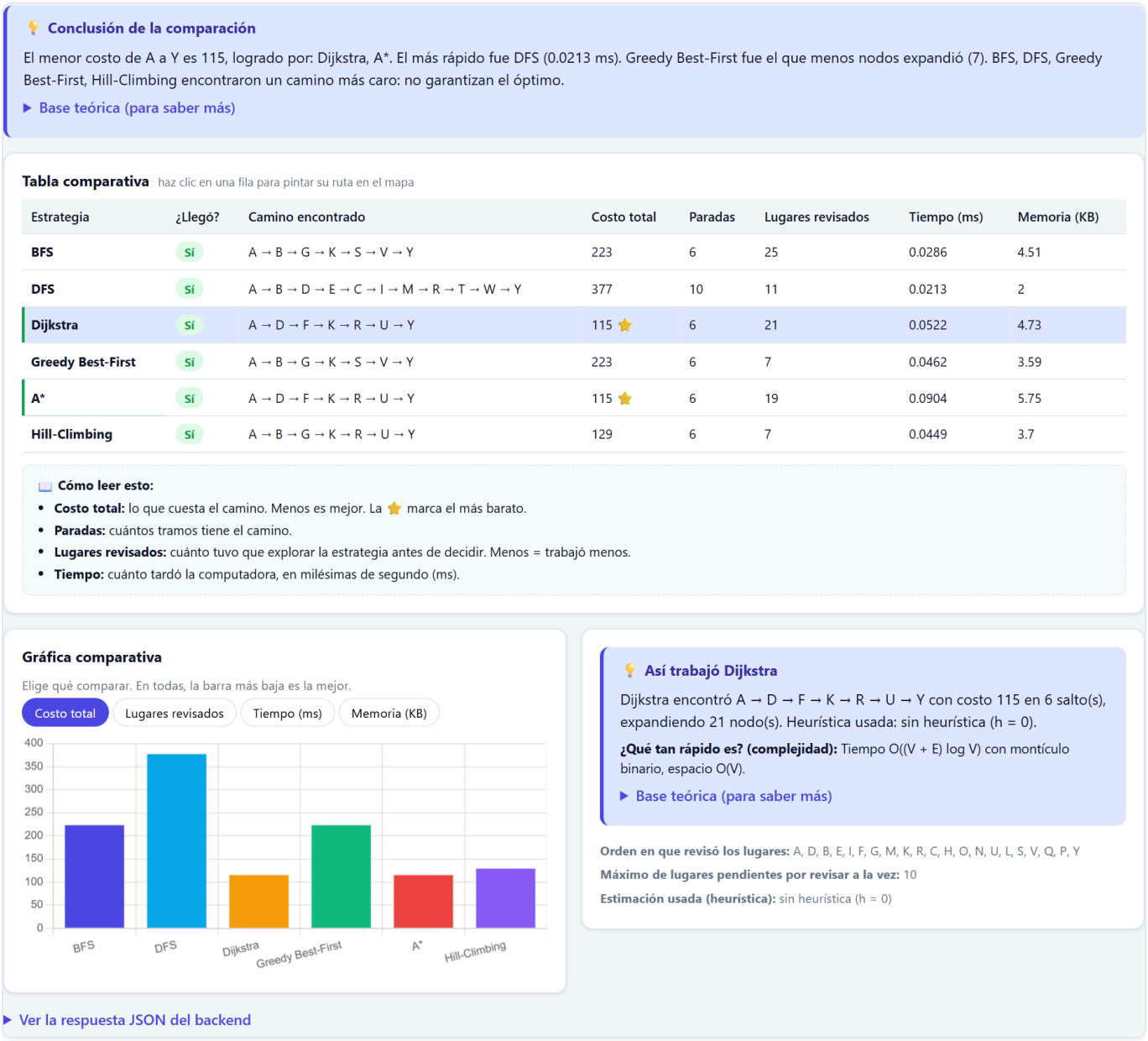


Figura 17. Conclusión, tabla comparativa, gráfica y detalle del algoritmo seleccionado.

- **Conclusión de la comparación**: quién encontró la ruta más barata, quién fue más rápido y quién revisó menos lugares.
- **Tabla comparativa**: camino, costo total (la ⭐ marca el más barato), paradas, lugares revisados, tiempo y memoria. **Haz clic en una fila** para pintar esa ruta en el mapa.
- **Gráfica comparativa**: elige con los botones qué métrica comparar. En todas, la barra más baja es la mejor.
- **Así trabajó…**: detalle del algoritmo seleccionado, el orden en que revisó los lugares y la heurística usada.

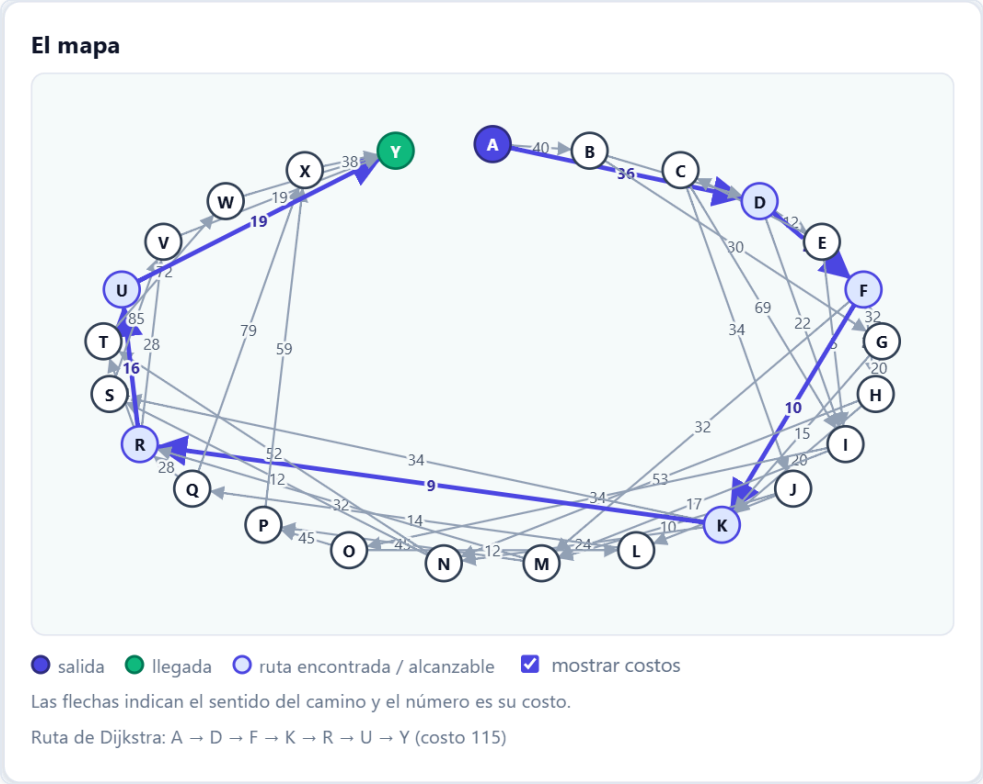


Figura 18. Al hacer clic en Dijkstra se pinta su ruta: A → D → F → K → R → U → Y, costo 115.

📌 Prueba sugerida: haz clic en **BFS** y luego en **Dijkstra**. El camino con menos paradas no siempre es el más barato.

5.4 📍 ¿Hasta dónde llego? (alcance)

Calcula a qué lugares puedes llegar desde un punto, por ejemplo «si solo tengo 120 de presupuesto» o «si solo puedo hacer 2 transbordos».

1. Cambia a la sub-pestaña 📍 **¿Hasta dónde llego?**.
2. Elige el **origen**.
3. Opcionalmente escribe un **Presupuesto máximo** (costo) y/o un **Máximo de paradas** (saltos). Vacío = sin límite.
4. Presiona Calcular alcance.

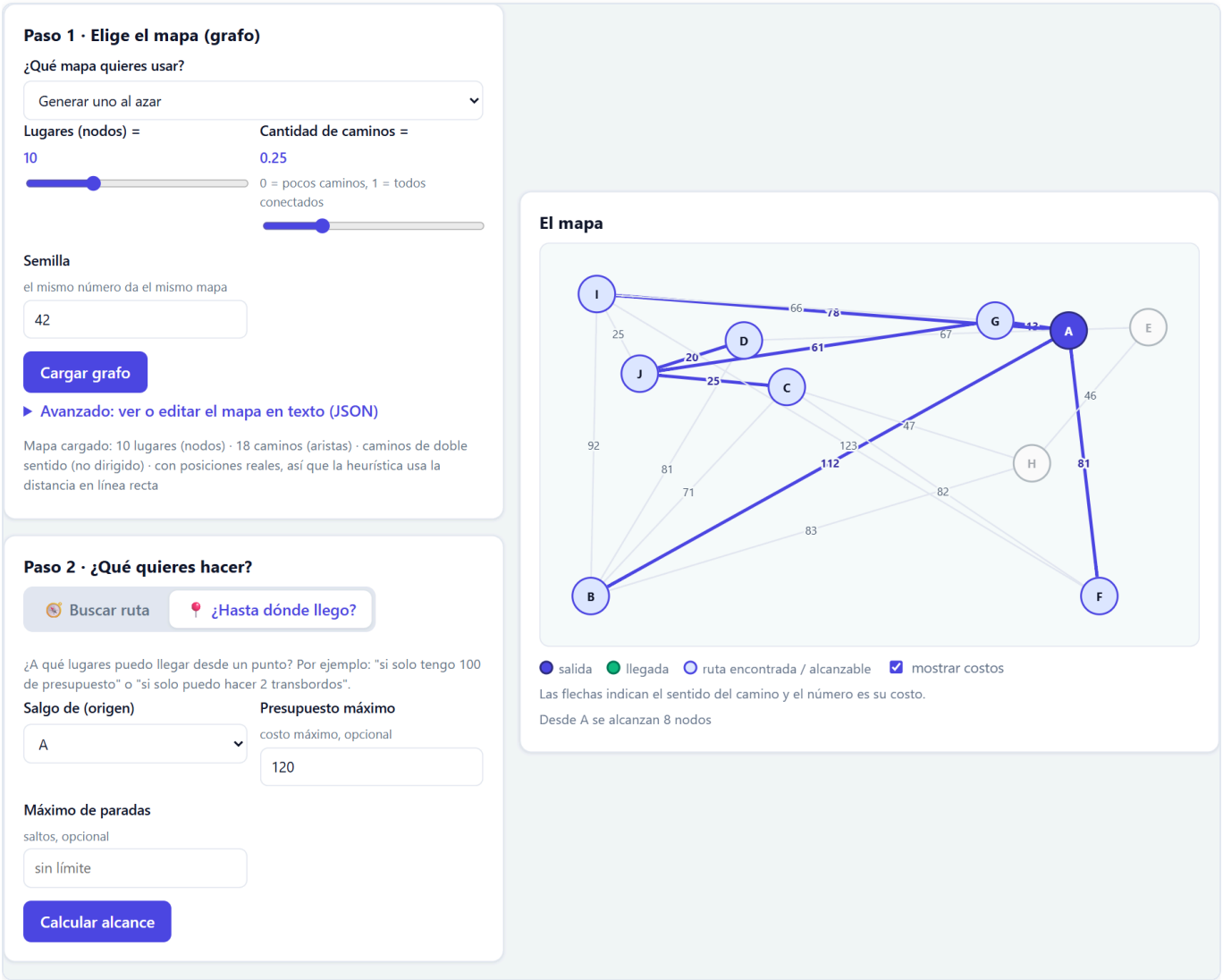


Figura 19. Lugares alcanzables desde A con presupuesto 120. Los grises quedan fuera de alcance.

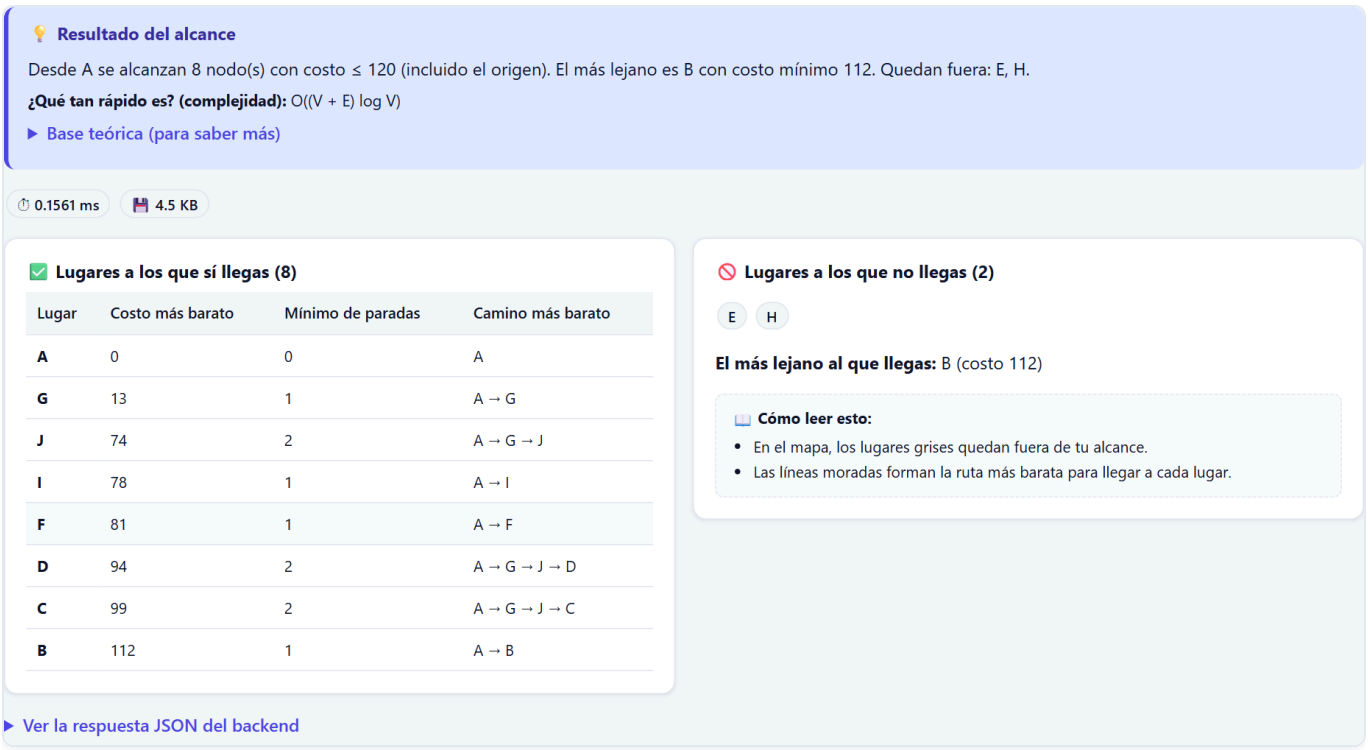


Figura 20. Tabla de lugares alcanzables con su costo más barato, mínimo de paradas y camino, y lista de los que no se alcanzan.

6. Módulo 3 · Escalabilidad y benchmark

Tiene tres sub-pestañas: 🔢 **Ordenar números**, 🚚 **Agente viajero** y 📈 **¿Qué pasa si crece?**.

6.1 Competencia de algoritmos de ordenamiento

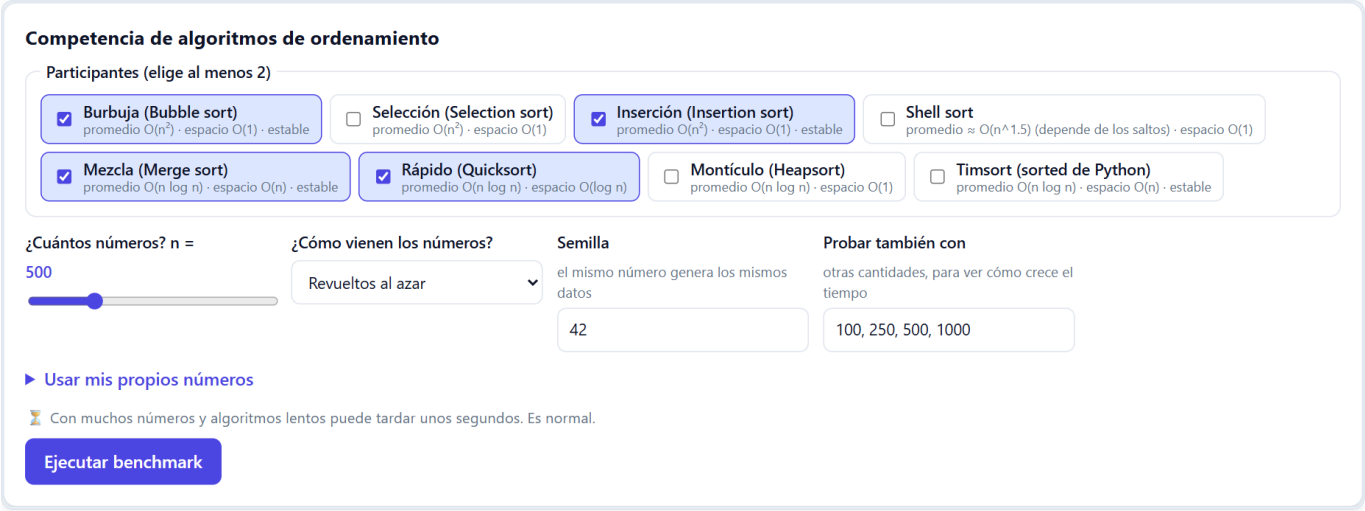


Figura 21. Formulario del benchmark de ordenamiento.

1. Marca al menos **2 participantes**: Burbuja, Selección, Inserción, Shell, Mezcla (Merge), Rápido (Quick), Montículo (Heap) y Timsort. Cada uno muestra su complejidad teórica.
2. Mueve el deslizador **¿Cuántos números?** (10 a 2000).
3. Elige **¿Cómo vienen los números?**: revueltos, ya ordenados, al revés, casi ordenados o con muchos repetidos.
4. En **Probar también con** escribe otras cantidades (por ejemplo `100, 250, 500, 1000`) para dibujar la curva de crecimiento.
5. Opcional: en **Usar mis propios números** escribe tu lista separada por comas.
6. Presiona **Ejecutar benchmark**.

> ⏳ Con muchos números y algoritmos lentos (Burbuja, Selección, Inserción) puede tardar varios segundos. Es normal.



Figura 22. Resultado del benchmark con 500 números revueltos y 4 algoritmos.

- **¿Quién ordenó más rápido?**: ranking con medallas 🥇 🥈 🥉 y el tiempo de cada uno.
- **Gráfica comparativa**: elige tiempo, comparaciones, movimientos o memoria.
- **Los datos: antes y después**: los primeros 40 números para comprobar que quedaron ordenados.
- **¿Qué pasa cuando hay más números?**: las líneas que se disparan hacia arriba son algoritmos $O(n^2)$; las casi planas son O(n log n).
- **Tabla de resultados**: incluye **¿Ordenó bien?**, si es **estable** y su complejidad teórica.

6.2 🚚 Agente viajero

Un repartidor debe visitar **todas las ciudades una sola vez** y volver al inicio recorriendo la menor distancia. Se compara el método **exacto (Held-Karp)**, que siempre encuentra la mejor ruta, contra el **voraz (vecino más cercano)**, que es rapidísimo pero puede equivocarse.

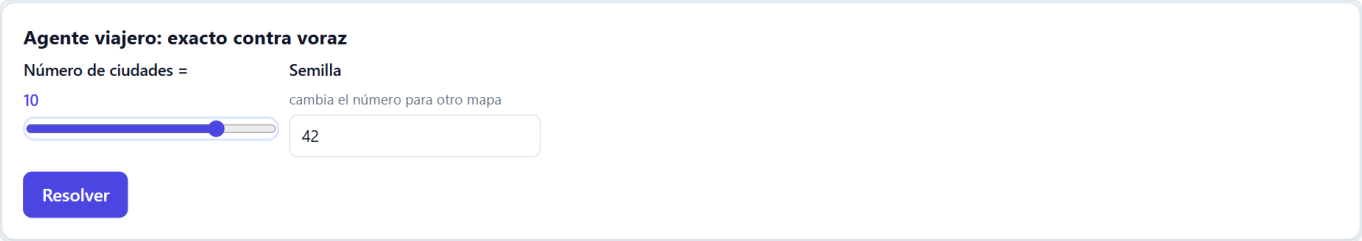


Figura 23. Elige el número de ciudades (3 a 12) y la semilla del mapa.

1. Mueve el deslizador **Número de ciudades**.
2. Cambia la **Semilla** si quieres otro mapa.
3. Presiona Resolver.

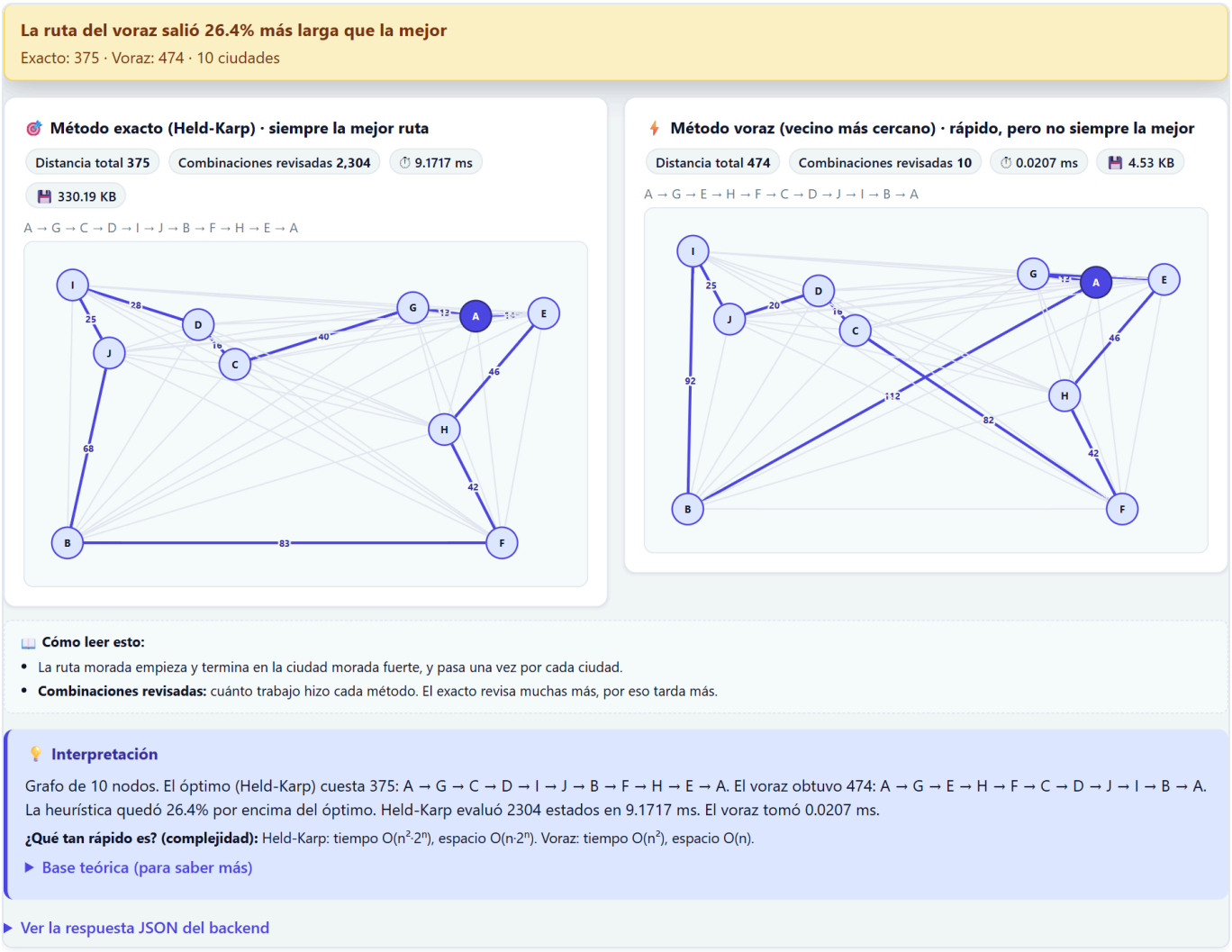


Figura 24. Con 10 ciudades el exacto encuentra una ruta de 375 y el voraz una de 474 (26.4 % más larga), pero el voraz tarda cientos de veces menos.

6.3 📈 ¿Qué pasa si crece? (P vs NP)

Resuelve el agente viajero con cada vez más ciudades para ver cómo crece el trabajo.

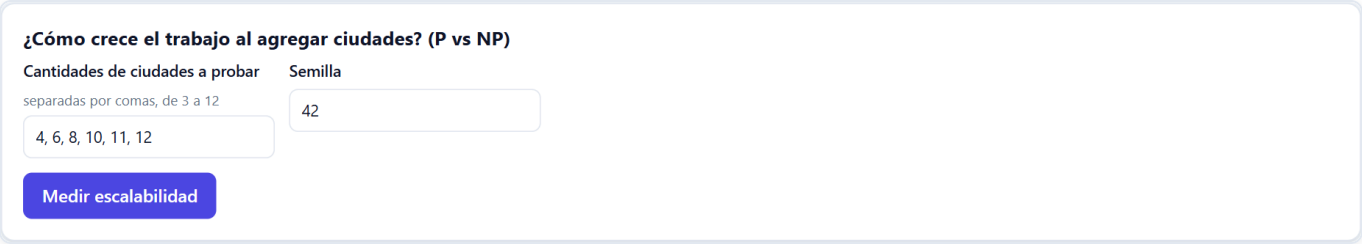


Figura 25. Cantidades de ciudades a probar (de 3 a 12, separadas por comas).

1. Deja los tamaños de ejemplo o escribe otros entre 3 y 12.
2. Presiona Medir escalabilidad.
3. Alterna entre **Tiempo** y **Memoria**, y activa **Escala logarítmica**: con ella un crecimiento exponencial se ve como una línea recta.

Interpretación

Se midieron grafos de 4 a 12 nodos. El tiempo de Held-Karp creció unas 757 veces y su memoria pasó de 1.88 KB a 2765.41 KB: crecimiento exponencial. El voraz pasó de 0.0076 ms a 0.0286 ms: crecimiento polinómico.

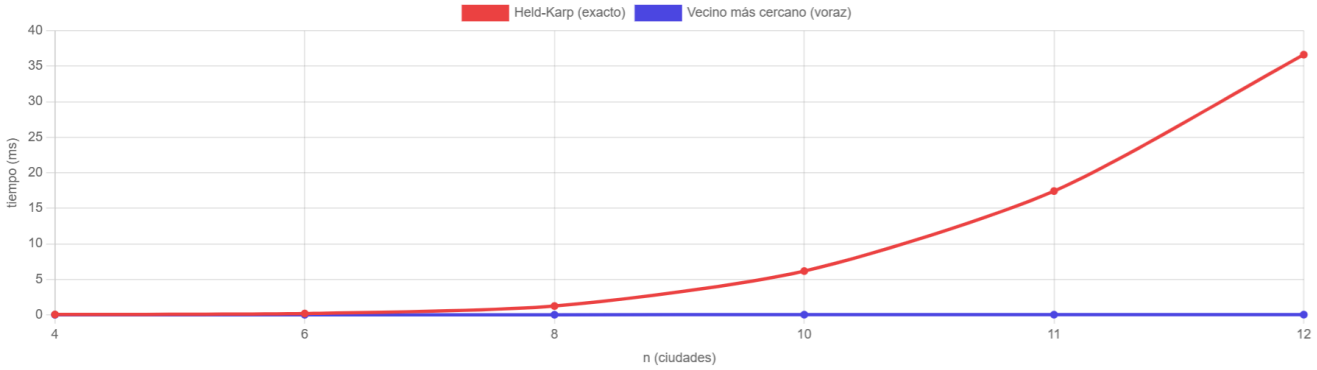
¿Qué tan rápido es? (complejidad): Held-Karp: tiempo $O(n^2 \cdot 2^n)$, espacio $O(n \cdot 2^n)$. Voraz: tiempo $O(n^2)$, espacio $O(n)$.

▶ Base teórica (para saber más)

Exacto contra voraz al agregar ciudades

Tiempo (ms) | Memoria (KB)

☐ Escala logarítmica: con ella, un crecimiento exponencial se ve como una línea recta



Cómo leer esto:

- Abajo está el número de ciudades; a la izquierda, el tiempo o la memoria.
- La línea **roja** (exacto) se dispara: crecimiento exponencial, típico de los problemas NP-difíciles.
- La línea **azul** (voraz) casi no sube: crecimiento polinómico, de la clase P.

Tabla de mediciones

Ciudades	Tiempo exacto (ms)	Tiempo voraz (ms)	Memoria exacto (KB)	Memoria voraz (KB)	Combinaciones (exacto)	Distancia exacta	Distancia voraz	¿Cuánto peor fue
4	0.0484	0.0076	1.88	0.92	12	253	259	2.37%
6	0.1885	0.0124	6.32	2	80	320	337	5.31%
8	1.2432	0.0163	48.98	4	448	327	418	27.83%
10	6.1634	0.0244	330.19	4.53	2,304	375	474	26.4%
11	17.4272	0.0238	841.33	4.98	5,120	389	491	26.22%
12	36.6309	0.0286	2,765.41	7.5	11,264	400	502	25.5%

▶ Ver la respuesta JSON del backend

Figura 26. La línea roja (exacto) se dispara de forma exponencial; la azul (voraz) casi no sube.

7. Mensajes de error

Si un dato está mal, el laboratorio **no se rompe**: muestra un recuadro rojo debajo del formulario y un aviso flotante en la esquina inferior derecha que dice qué corregir.

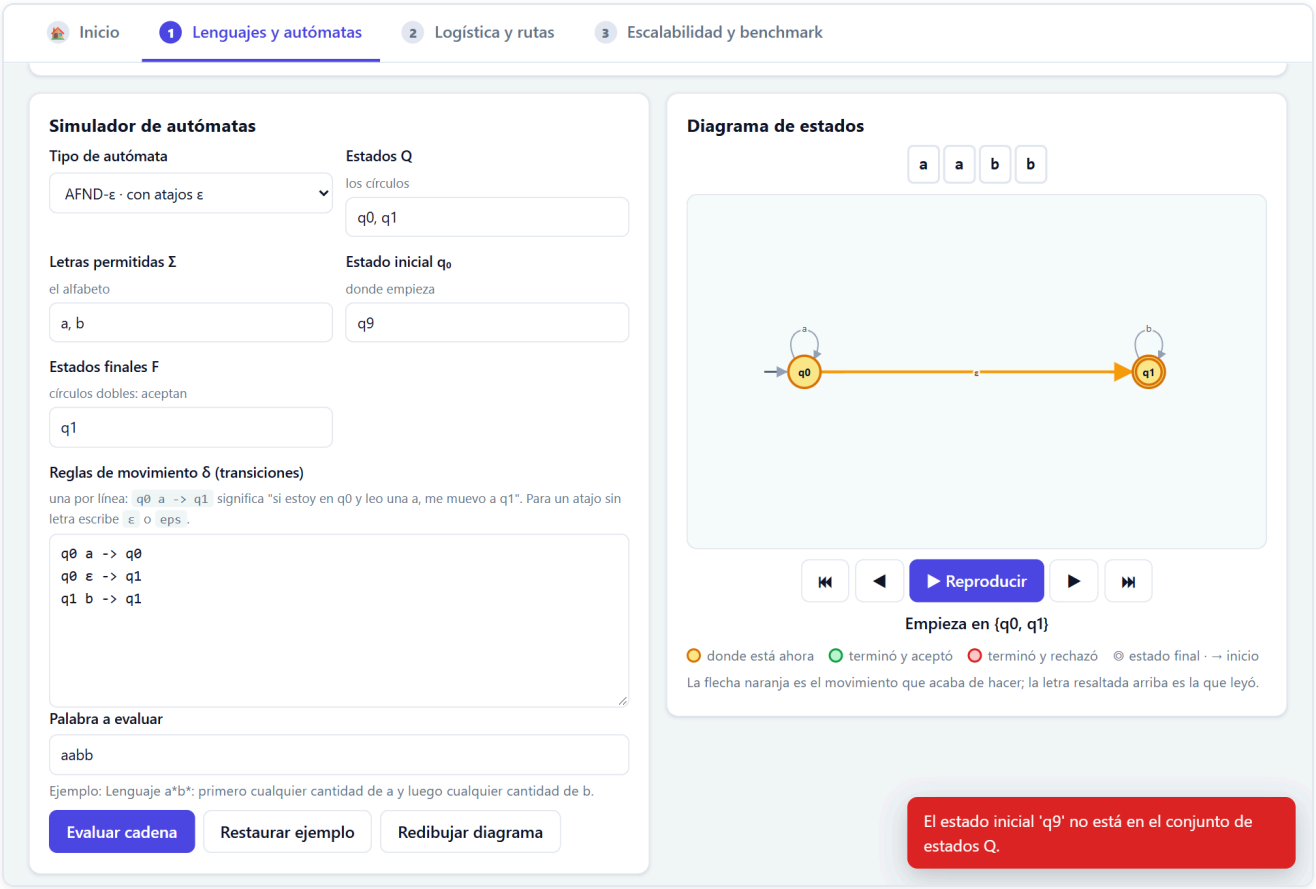


Figura 27. Se escribió el estado inicial «q9», que no existe en Q. El aviso indica el problema.

No se pudo completar la operación. El estado inicial 'q9' no está en el conjunto de estados Q.

Figura 28. Recuadro de error bajo el formulario.

Mensaje (ejemplo)	Causa	Cómo corregirlo
El estado inicial 'q9' no está en el conjunto de estados Q.	El estado inicial, un final o un destino no aparece en Q.	Agrega el estado a **Estados Q** o corrige el nombre.
La transición δ(q0, c) usa un símbolo que no está en Σ.	Una regla usa una letra que no está en el alfabeto.	Agrega la letra a **Letras permitidas Σ**.
El origen 'Z' no pertenece al grafo.	Se escribió un nodo que no está en el mapa.	Elige un lugar de la lista o recarga el grafo.
Selecciona al menos 2 algoritmos distintos.	En el benchmark solo hay un participante.	Marca otro algoritmo.
Cada tamaño debe estar entre 3 y 12 nodos.	Un tamaño del TSP está fuera de rango.	Usa valores de 3 a 12.
Backend sin conexión	El servidor no está corriendo.	Arranca `uvicorn` (sección 2.2).

8. Preguntas frecuentes

¿Cómo escribo la palabra vacía?

Escribe `ε` (épsilon). En las reglas de un AFND-ε también puedes escribir `eps`.

¿Por qué L* o el complemento no muestran todas las palabras?

Porque son conjuntos infinitos. Solo se muestran las palabras de hasta **k** letras; sube el deslizador para ver más.

¿Por qué los tiempos cambian cada vez que ejecuto?

El tiempo depende de lo que esté haciendo tu computadora en ese momento. El laboratorio repite la medición y se queda con el mejor tiempo, pero siempre habrá pequeñas variaciones. Lo importante es la **tendencia**.

¿Por qué el agente viajero solo acepta hasta 12 ciudades?

El método exacto crece de forma exponencial: con 12 ciudades ya revisa más de 11,000 combinaciones y usa casi 3 MB de memoria. Con unas pocas ciudades más tardaría demasiado. Esa es justamente la lección de P contra NP.

¿Por qué el benchmark de ordenamiento acepta como máximo 2000 números?

Medir la memoria hace más lentos a los algoritmos $O(n^2)$. Con 2000 números y los 8 algoritmos la respuesta ya tarda unos 18 segundos.

¿Dónde veo exactamente lo que respondió el servidor?

Al final de cada resultado, despliega **Ver la respuesta JSON del backend**. También puedes probar la API directamente en `http://localhost:8000/docs`.

La página aparece en blanco o sin gráficas.

Ábrela desde `http://localhost:5500` (no con doble clic) y revisa tu conexión a internet para que cargue Chart.js.

9. Glosario

Término	Significado
Alfabeto (Σ)	Las letras o símbolos permitidos. Ejemplo: {a, b} o {0, 1}.
Cadena o palabra (w)	Una secuencia de letras del alfabeto, como `abba`.
ε (épsilon)	La palabra vacía: una palabra sin ninguna letra.
Lenguaje (L)	Un conjunto de palabras, como {a, ab, abb}.
Cerradura de Kleene (L*)	Todas las palabras que se forman concatenando palabras de L cero o más veces.
Autómata	Máquina que lee una palabra letra por letra y al final la acepta o la rechaza.
AFD / AFND / AFND-ε	Autómata determinista (un solo camino), no determinista (varios caminos a la vez) y no determinista con transiciones vacías ε.
Estado / estado final	Cada círculo del diagrama / círculo doble: si la máquina termina ahí, acepta.
Transición (δ)	Una flecha: «si estoy en q0 y leo una a, me muevo a q1».
Cerradura-ε	Los estados a los que se llega usando solo flechas ε.
Grafo	Un mapa de puntos (nodos) unidos por líneas (aristas).
Grafo dirigido	Las aristas tienen sentido, como calles de un solo sentido.
Peso o costo	El número sobre cada arista: lo que cuesta recorrerla.
Heurística	Una estimación de qué tan lejos falta para llegar al destino.
Nodos expandidos / lugares revisados	Cuántos lugares tuvo que explorar un algoritmo antes de decidir.
Algoritmo	Una receta: una lista de pasos para resolver un problema.
Complejidad O(n)	Cómo crece el trabajo cuando crecen los datos. $O(n^2)$ crece mucho más rápido que O(n log n).
Ordenamiento estable	Si dos números son iguales, conserva el orden en que venían.
Agente viajero (TSP)	Visitar todas las ciudades una vez y volver al inicio con la menor distancia.
P y NP	P: problemas que una computadora resuelve rápido. Los NP-difíciles, como el agente viajero, se vuelven lentísimos al crecer.